Министерство образования Республики Беларусь
Учреждение Образования
«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Лабораторная работа № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»

Выполнил
студент гр. 439401
Ермалович М.С.

Проверил
Силич С.С.

Минск 2026

Цель работы: разработать утилиту «Высота и длина». Пользователь вводит длину, единицу измерения, в которой введена длина, и единицу измерения, в которую нужно конвертировать эту длину. Поддерживаемые единицы измерения: дюйм, ярд, сантиметр, метр, американский таракан, шея жирафа, самая длинная змея, человеческий язык, футбольное поле. На главной форме размещена кнопка «Ввести данные», по нажатию на которую открывается дополнительное окно для ввода информации. Последние введенные данные запоминаются в главной форме и восстанавливаются при повторном вводе. После этого на главной форме отображается результат вычислений. Если введены некорректные данные – показывается сообщение об ошибке.

Теоретические сведения

Model-View-Controller (MVC) – это фундаментальный шаблон проектирования, предназначенный для разделения бизнес-логики приложения (компонентов работы с данными) от его визуального представления.

Согласно классической концепции, архитектура приложения разделяется на три изолированных класса:

- Модель (Model) – управляет внутренним состоянием, данными и бизнес-логикой домена приложения. Она отвечает на запросы информации о своем состоянии и обрабатывает инструкции по его изменению. Модель является полностью независимым компонентом: она ничего не знает ни о Представлении, ни о Контроллере.
- Представление (View) – отвечает за графический интерфейс (GUI), визуальное отображение данных пользователю и презентационную логику.
- Контроллер (Controller) – перехватывает действия пользователя (нажатия клавиш, клики мыши), интерпретирует их и вызывает соответствующие методы Модели для изменения её состояния.

Активная и пассивная модели в MVC

В зависимости от способа синхронизации данных и интерфейса, автор паттерна Стив Бурбек (Steve Burbeck) выделяет две ключевые модификации MVC:

- Пассивная модель (Passive Model):

Используется в архитектурах, где Контроллер является единственным источником изменений Модели. В этом сценарии Модель полностью автономна и не способна сообщать о смене своего состояния. Контроллер сначала модифицирует Модель, а затем самостоятельно и принудительно дает команду Представлению обновить отображение на экране. Классическим примером такой схемы является стандартный протокол HTTP в веб-приложениях.

- Активная модель (Active Model):

Применяется, когда состояние Модели может меняться без прямого участия текущего Контроллера (например, при поступлении внешних данных,

вычислениях по таймеру или многопоточной обработке). Так как только сама Модель способна зафиксировать изменения своего внутреннего состояния, она обязана уведомить интерфейс о необходимости синхронизации. Чтобы сохранить независимость Модели от конкретных окон интерфейса, активная модель реализуется через паттерн Observer («Наблюдатель»), также известный как схема *«издатель-подписчик» (publish-subscribe)*. Представления регистрируются в Модели в качестве наблюдателей. При изменении бизнес-данных Модель циклически обходит список подписчиков и автоматически генерирует для них сигналы оповещения, изолируя логику расчета от визуального кода.

Современная IDE (на примере связки VS Code и инструментария GCC/MinGW) – это комплекс программных средств для автоматизации разработки, состоящий из следующих компонентов:

- Текстовый редактор кода – оболочка с функциями подсветки синтаксиса и структурирования исходных файлов.
- Компилятор и Линкер (g++.exe) – транслятор кода C++ в машинные объектные модули и компоновщик их с системными библиотеками в файл .exe.
- Система автоматизации сборки (tasks.json) – диспетчер, управляющий последовательностью компиляции и передачей флагов оптимизации.
- Отладчик (gdb.exe) – инструмент для пошагового выполнения программы, установки точек останова (breakpoints) и инспекции переменных в памяти.
- Менеджер расширений – подсистема интеграции плагинов (например, для поддержки UML и SysMLv2).

Основные приемы работы с редактором кода

Эффективное написание кода и снижение числа ошибок обеспечиваются следующими встроенными технологиями редактора:

- Автодополнение (IntelliSense) – контекстные подсказки имен функций, классов и типов данных на лету при вводе первых букв.
- Подсветка синтаксиса – цветовое выделение ключевых слов языка, типов данных, строк и комментариев для улучшения читаемости кода.
- Фоновый статический анализ (Линтинг) — мгновенное обнаружение и подчеркивание синтаксических ошибок (например, несовместимости типов в WinAPI) еще до запуска компиляции.
- Навигация и рефакторинг – быстрое перемещение к объявлению функций (клавиша F12) и безопасное массовое переименование переменных по всему проекту.

Расширения для UML / SysMLv2

В MS Visual Studio Code для работы с диаграммами UML использовано расширение «PlantUML», для работы с моделями SysMLv2 – расширение «SysML v2», предоставляющее подсветку синтаксиса и предпросмотр

диаграмм системного моделирования. Для IntelliJ IDEA и Eclipse аналогами являются плагины «PlantUML integration» и «Eclipse Papyrus» соответственно.

Код программы

```
#include <windows.h>
#include <string>
#include <stdexcept>

using namespace std;

// ========================================
// 1. МОДЕЛЬ (Model) - Бизнес-логика
// ========================================

const int UNIT_COUNT = 9;

const wchar_t* unitNames[UNIT_COUNT] = {
L"Дюйм", L"Ярд", L"Сантиметр", L"Метр", L"Американский таракан",
L"Шея жирафа", L"Самая длинная змея", L"Человеческий язык", L"Футбольное
поле"
};

// Коэффициенты перевода единицы измерения в сантиметры
const double unitToCm[UNIT_COUNT] = {
2.54, 91.44, 1.0, 100.0, 4.0, 200.0, 743.0, 10.1, 10500.0
};

class LengthConverterModel {
private:
double sourceValue = 0.0;
int sourceUnitIndex = 2;
int targetUnitIndex = 3;
double result = 0.0;
bool hasData = false;
public:
HWND viewWindow = nullptr;

void updateData(double value, int srcIdx, int tgtIdx) {
if (value <= 0 || srcIdx < 0 || srcIdx >= UNIT_COUNT || tgtIdx < 0 ||
tgtIdx >= UNIT_COUNT) {
throw invalid_argument("Введены некорректные данные!");
}
sourceValue = value;
sourceUnitIndex = srcIdx;
targetUnitIndex = tgtIdx;
result = value * unitToCm[srcIdx] / unitToCm[tgtIdx];
hasData = true;
if (viewWindow) {
PostMessageW(viewWindow, WM_USER + 1, 0, 0);
}
}

bool getHasData() const { return hasData; }
double getSourceValue() const { return sourceValue; }
int getSourceUnitIndex() const { return sourceUnitIndex; }
int getTargetUnitIndex() const { return targetUnitIndex; }
double getResult() const { return result; }
};

LengthConverterModel g_model;

// ========================================
```

```
      // 2. КОНТРОЛЛЕР (Controller)
      // ========================================

      class LengthConverterController {
      public:
      bool trySaveData(const wstring& valueStr, int srcIdx, int tgtIdx,
wstring& outError) {
      try {
      if (valueStr.empty()) {
      throw invalid_argument("Пустое значение!");
      }
      size_t pos = 0;
      double value = stod(valueStr, &pos);
      if (pos != valueStr.size()) {
      throw invalid_argument("Некорректный формат числа!");
      }
      g_model.updateData(value, srcIdx, tgtIdx);
      return true;
      }
      catch (...) {
      outError = L"Ошибка ввода! Проверьте корректность введенной длины и
единиц измерения.";
      return false;
      }
      }
      };

      LengthConverterController g_controller;

      HWND hValue, hSourceUnit, hTargetUnit;
      HWND hLblInput, hLblResult;

      // Форматирование вещественного числа с двумя знаками после запятой
      wstring formatDouble(double n) {
      wchar_t buf[64];
      swprintf(buf, 64, L"%.2f", n);
      return wstring(buf);
      }

      // ========================================
      // 3. ПРЕДСТАВЛЕНИЕ (View) - Интерфейс
      // ========================================

      LRESULT CALLBACK InputWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{
      switch (msg) {
      case WM_CREATE: {
      CreateWindowW(L"Static", L"Длина:", WS_VISIBLE | WS_CHILD,
      10, 15, 100, 20, hwnd, NULL, NULL, NULL);
      hValue = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD | WS_BORDER,
      120, 12, 100, 22, hwnd, NULL, NULL, NULL);

      CreateWindowW(L"Static", L"Единица измерения (откуда):", WS_VISIBLE |
WS_CHILD,
      10, 50, 200, 20, hwnd, NULL, NULL, NULL);
      hSourceUnit = CreateWindowW(L"Combobox", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER | CBS_DROPDOWNLIST,
      10, 72, 260, 200, hwnd, NULL, NULL, NULL);

      CreateWindowW(L"Static", L"Единица измерения (куда):", WS_VISIBLE |
WS_CHILD,
      10, 105, 200, 20, hwnd, NULL, NULL, NULL);
      hTargetUnit = CreateWindowW(L"Combobox", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER | CBS_DROPDOWNLIST,
```

```
      10, 127, 260, 200, hwnd, NULL, NULL, NULL);

      for (int i = 0; i < UNIT_COUNT; i++) {
      SendMessageW(hSourceUnit, CB_ADDSTRING, 0, (LPARAM)unitNames[i]);
      SendMessageW(hTargetUnit, CB_ADDSTRING, 0, (LPARAM)unitNames[i]);
      }

      if (g_model.getHasData()) {
      SetWindowTextW(hValue, formatDouble(g_model.getSourceValue()).c_str());
      SendMessageW(hSourceUnit, CB_SETCURSEL, g_model.getSourceUnitIndex(),
0);
      SendMessageW(hTargetUnit, CB_SETCURSEL, g_model.getTargetUnitIndex(),
0);
      }
      else {
      SendMessageW(hSourceUnit, CB_SETCURSEL, 2, 0);
      SendMessageW(hTargetUnit, CB_SETCURSEL, 3, 0);
      }

      CreateWindowW(L"Button", L"Сохранить", WS_VISIBLE | WS_CHILD,
      95, 165, 100, 32, hwnd, (HMENU)1, NULL, NULL);
      break;
      }
      case WM_COMMAND: {
      if (LOWORD(wp) == 1) {
      wchar_t buf[64];
      GetWindowTextW(hValue, buf, 64);
      int srcIdx = (int)SendMessageW(hSourceUnit, CB_GETCURSEL, 0, 0);
      int tgtIdx = (int)SendMessageW(hTargetUnit, CB_GETCURSEL, 0, 0);
      wstring error;
      if (g_controller.trySaveData(buf, srcIdx, tgtIdx, error)) {
      DestroyWindow(hwnd);
      }
      else {
      MessageBoxW(hwnd, error.c_str(), L"Ошибка", MB_OK | MB_ICONERROR);
      }
      }
      break;
      }
      default:
      return DefWindowProcW(hwnd, msg, wp, lp);
      }
      return 0;
      }

      LRESULT CALLBACK MainWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{
      switch (msg) {
      case WM_CREATE: {
      g_model.viewWindow = hwnd;
      CreateWindowW(L"Button", L"Ввести данные", WS_VISIBLE | WS_CHILD,
      20, 20, 150, 32, hwnd, (HMENU)1, NULL, NULL);
      hLblInput = CreateWindowW(L"Static", L"Введено: -", WS_VISIBLE |
WS_CHILD,
      20, 75, 440, 20, hwnd, NULL, NULL, NULL);
      hLblResult = CreateWindowW(L"Static", L"Результат: не рассчитано",
      WS_VISIBLE | WS_CHILD, 20, 107, 440, 20, hwnd, NULL, NULL, NULL);
      break;
      }
      case WM_COMMAND: {
      if (LOWORD(wp) == 1) {
      CreateWindowExW(0, L"InputWindowClass", L"Ввод данных",
      WS_OVERLAPPEDWINDOW | WS_VISIBLE,
      CW_USEDEFAULT, CW_USEDEFAULT, 300, 250, hwnd, NULL, NULL, NULL);
```

```
        }
        break;
        }
        case WM_USER + 1: {
        wstring inputText = L"Введено: " +
formatDouble(g_model.getSourceValue()) + L" "
        + unitNames[g_model.getSourceUnitIndex()];
        wstring resultText = L"Результат: " + formatDouble(g_model.getResult())
+ L" "
        + unitNames[g_model.getTargetUnitIndex()];
        SetWindowTextW(hLblInput, inputText.c_str());
        SetWindowTextW(hLblResult, resultText.c_str());
        break;
        }
        case WM_DESTROY:
        PostQuitMessage(0);
        break;
        default:
        return DefWindowProcW(hwnd, msg, wp, lp);
        }
        return 0;
        }

        int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {
        WNDCLASSW wcMain = { 0 };
        wcMain.lpfnWndProc = MainWndProc;
        wcMain.hInstance = hInst;
        wcMain.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcMain.lpszClassName = L"MainWindowClass";
        RegisterClassW(&wcMain);

        WNDCLASSW wcInput = { 0 };
        wcInput.lpfnWndProc = InputWndProc;
        wcInput.hInstance = hInst;
        wcInput.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcInput.lpszClassName = L"InputWindowClass";
        RegisterClassW(&wcInput);

        HWND hwnd = CreateWindowExW(
        0, L"MainWindowClass", L"LengthConverterAPP",
        WS_OVERLAPPEDWINDOW | WS_VISIBLE,
        CW_USEDEFAULT, CW_USEDEFAULT, 490, 190,
        NULL, NULL, hInst, NULL
        );

        MSG msg = { 0 };
        while (GetMessageW(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
        }
        return 0;
        }
```

Вывод программы



Рисунок 1 – Главный экран

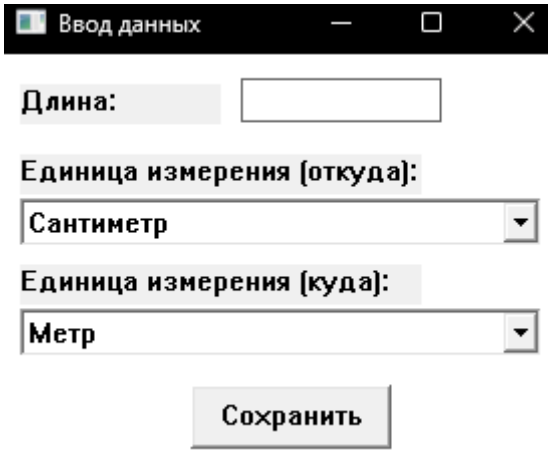


Рисунок 2 – Окно ввода данных

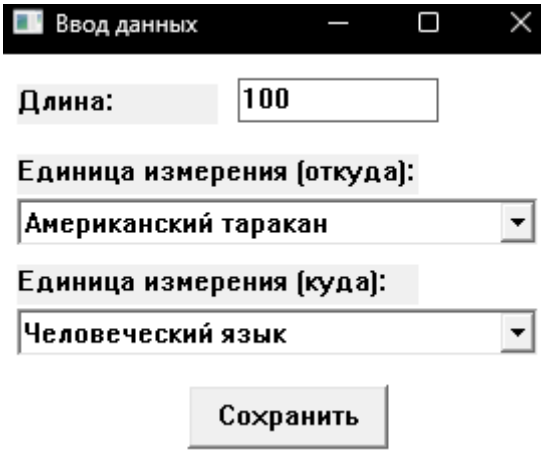


Рисунок 3 – Окно ввода данных после заполнения полей



Рисунок 4 – Главный экран после выполнения расчета

Заключение

В ходе выполнения лабораторной работы было разработано графическое приложение-утилита «Высота и длина», выполняющее конвертацию длины между единицами измерения: дюйм, ярд, сантиметр, метр, американский таракан, шея жирафа, самая длинная змея, человеческий язык и футбольное поле. Реализован механизм проверки корректности введённых данных и восстановления ранее введённых значений при повторном открытии окна ввода.

Приложение спроектировано по паттерну MVC с активной моделью: синхронизация данных и интерфейса реализована через системные сообщения Windows (PostMessageW/WM_USER) на основе паттерна «Наблюдатель», что позволяет Модели самостоятельно уведомлять Представление об изменении своего состояния без прямого вызова со стороны Контроллера. Использование низкоуровневого Windows API для построения оконного интерфейса позволило полностью выполнить требование об использовании нестандартных элементов управления (списков ComboBox) и отказе от высокоуровневых фреймворков. Разработка, сборка автономного .exe файла и пошаговая отладка проекта выполняются в среде MS Visual Studio Code с компилятором MinGW (g++/gdb).